



One World Project Audit Report

Version 1.0

Shikhar Agarwal

December 6, 2024

One World Project Audit Report

Shikhar Agarwal

Dec 6, 2024

Prepared by: Shikhar Agarwal (<https://profiles.cyfrin.io/u/shikhar229169>)

Lead Auditors: - Shikhar Agarwal

Table of Contents

- About the Project
- Disclaimer
- Risk Classification
- Audit Details
 - Timeline
 - Sponsor
 - Scope
 - Roles
 - Issues found
- Findings
 - High Risk Findings
 - Medium Risk Findings
 - Low Risk Findings

About the Project

One World Project is a dynamic DAO marketplace where users can actively participate in decentralized organizations. This repository contains smart contracts for creating and managing DAO memberships

using ERC1155 tokens. The contracts facilitate the creation and management of DAO memberships. Each DAO membership is represented by an ERC1155 token with different tiers.

Twitter: https://x.com/1W_Project
Website: <https://www.oneworldproject.io/>
See more contest details here - <https://codehawks.cyfrin.io/c/2024-11-one-world>

Disclaimer

As the sole auditor all efforts have been made to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Timeline

Nov 6th, 2024 - Nov 13th, 2024

One World

Scope

Roles

DAO Creator: A user that creates a DAO.

Issues found

Severity	Count of Findings
High	1
Medium	2
Low	1

Findings

High Risk Findings

H-01. No verification of OWPIidentity token in MembershipFactory, MembershipERC1155 allows users to interact with the protocol without going through the KYC process

Medium Risk Findings

M-01. Use of CREATE method is suspicious of reorg attack in MembershipFactory::createNewDAOMembership

M-02. MembershipERC1155::setURI function being controlled by DAO creator can be maliciously called to change the URI to perform XSS attacks

Low Risk Findings

L-01. No enforcement of max members for sponsored DAO allows more users to join crossing the max cap

High Risk Findings

H-01. No verification of OWPIidentity token in MembershipFactory, MembershipERC1155 allows users to interact with the protocol without going through the KYC process

Summary

- The protocol performs a KYC process before allowing a user to interact with the protocol. After a successful KYC verification OWPIidentity token is minted for the user which serves as a proof on-chain that user has completed the KYC off-chain.
- Therefore all the functions which allows direct interaction of the user with the protocol should have a check related to verification of caller's kyc via the OWPIidentity contract but in the actual implementation no function in the protocol utilizes the OWPIidentity contract for the verification of user's kyc.
- As a result, it allows the non-kyced user's to also interact with the protocol and get to perform all the operations and get all privileges same as a kyced user.

Vulnerability Details

- The vulnerability is present in all the contracts for which it expects kyced user's to interact with them, but due to the missing implementation of checking a user's KYC via the OWPIIdentity contract, it allows non-kyced user's to interact with the protocol. Thus breaking the invariant to only allow kyced user's to participate in the protocol.
- OWPIIdentity contract serves as a medium to verify the identity of the user on-chain. Initially a user performs KYC off-chain and after successful results the user gets to mint the OWPIIdentity token. The user is able to call the mint function on the OWPIIdentity by getting the `MINTER_ROLE`'s signed transaction and executing it via NativeMetaTransaction.
- But due to the checks related to OWPIIdentity token not present on required contracts allows anyone to call the functions, thus it doesn't depend on whether user has done kyc or not, allowing non-kyced user's to interact with the protocol.
- Also, as the ERC1155 token represented in MembershipERC1155 is transferrable to anyone, this opens the door allowing a user to transfer the token to any user (specifically non-kyced users). This allows them to sell the memberships to non-kyced users, assuming that all other operations were corrected with OWPIIdentity verification.

Impact

- This allows anyone to execute functions such as creating DAO membership, joining a DAO, upgrading tier, etc.
- Also, allows users to perform membership transfers to non-kyced users, allowing them to sell memberships to non-kyced users.

Tools Used

Manual Review

Recommendations

- As ERC1155 token is used for OWPIIdentity, therefore for minting any arbitrary token id can be used, therefore it is necessary to maintain the tokenId minted for a user.
- After that add a check to verify that the caller has the ERC1155 tokenId minted to them during successful KYC in all the relevant functions such as `createNewDAOMembership`, `joinDAO`, `upgradeTier`, `claimProfit`, etc.
- This allows only KYCed users to perform necessary operations to interact with the protocol.

Medium Risk Findings

M-01. Use of CREATE method is suspicious of reorg attack in `MembershipFactory::createNewDAOMembership`

Relevant Github Link

<https://github.com/Cyfrin/2024-11-one-world/blob/main/contracts/dao/MembershipFactory.sol#L72>

Summary

- When a user wants to create their membership DAO contract, then they do it via `MembershipFactory::createNewDAOMembership` which deploys a dedicated contract for handling their DAO membership using CREATE method and the derivation of the addresses are fully dependent on the `MembershipFactory` contract's nonce.
- Therefore, making it susceptible to reorg attacks.

Vulnerability Details

- The vulnerability is present in the `MembershipFactory::createNewDAOMembership` which uses CREATE method for deployment of the dedicated `MembershipERC1155` contract and makes it prone to reorg attacks.
- Reorgs occurs mostly on chains such as Polygon. Polygon is seen to have the largest number of reorgs being happening and as the protocol is mentioned to be specifically deployed to Polygon thus making it vulnerable to reorg attacks. The biggest reorg on Polygon set it back to almost 120 blocks.
- The `sendProfit` function on `MembershipERC1155` contract allows to send the profit to the contract or creator depending on supply condition.
- Consider 2 users calling `createNewDAOMembership` followed by others calling `sendProfit` for funding their DAO, where the creator A's creation of contract suffering from block reorg would make the profit sent to their address the creator B's membership dao contract's profit due to creator A's contract address being given to creator B due to a reorg.

Proof of Concept

1. Alice calls `createNewDAOMembership` function and funderA sends profit to it via `sendProfit` function.

2. Bob has an active bot that observes the polygon blockchain and alerts whenever there is a reorg.
3. Upon getting alerted for a reorg, Bob calls the `createNewDAOMembership` function with the same currency token as Alice.
4. Thus `MembershipERC1155` contract is created with an address to which funderA sent the currency tokens.
5. Finally Alice's tx is executed but the `MembershipERC1155` is funded by funderA having Bob as its creator.

Impact

Funds are vulnerable to be stolen which are sent via `sendProfit` function.

Tools Used

Manual Review

Recommendations

Update the `MembershipFactory::createNewDAOMembership` function to deploy the `MembershipERC1155` contract via `CREATE2` with `salt` that includes `msg.sender`.

M-02. `MembershipERC1155::setURI` function being controlled by DAO creator can be maliciously called to change the URI to perform XSS attacks

Summary

- The `MembershipERC1155::setURI` function being controlled by the DAO creator would result in the creator changing it maliciously conducting XSS attacks and leads to allowing the malicious DAO creator to perform malicious tasks such as run a keylogger script to collect all inputs typed by a user including password or to create a fake Metamask pop up asking a user to sign a malicious transaction.

Vulnerability Details

- Any user can create their own membership DAO contract via calling `MembershipFactory::createNewDAOMembership`, and this deploys a `MembershipERC1155` contract dedicated for them.

- A [MembershipERC1155](#) contract specifies a DAO membership contract created by the DAO creator where members can join in.
- The [setURI](#) function of [MembershipERC1155](#) is specifically privileged only to the respective DAO creator of that contract which further opens the door for a DAO creator to maliciously change the URI and performing XSS attacks on the user's machine.
- The metadata for the tokens are fetched from the [MembershipERC1155::uri](#) function and would be rendered directly in the frontend of websites such as the protocol website or any marketplace where the user has listed their membership for trade. The maliciously manipulated URI will lead to further XSS attacks on user's machine such as creating fake metamask popups or running keylogger scripts.
- This will produce a reflected XSS on all websites that load the malicious image from the [uri](#) function.

Impact

A malicious DAO creator creating [MembershipERC1155](#) contract and changing the uri to a malicious javascript payload, they could get a stored XSS on all websites that render the malicious uri. This could allow the attacker to perform malicious actions such as running a keylogger script to collect all inputs typed by a user including his password or to create a fake Metamask pop up asking a user to sign a malicious transaction.

Tools Used

Manual Review

Recommendations

There are certain mitigations for this: - Make the uri non changeable. - If its allowed for the DAO creator to change the uri, then enforce certain restrictions on the uri to ensure that the user's input is properly sanitized to not include any malicious symbol in there. - Make the uri to be only changeable by the [EXTERNAL_CALLER](#).

Low Risk Findings

L-01. No enforcement of max members for sponsored DAO allows more users to join crossing the max cap

Summary

- It is mentioned by the sponsors of the protocol that: `Sponsored DAO's are max capped at 19825 / 7 tiers`.
- But there is no enforcement of the same in the `createNewDAOMembership` function which allows the DAO creator to set any arbitrary amount for the allowed number of members for their sponsored DAO, this violates the invariant to keep the Sponsored DAO's are max capped at 19825 / 7 tiers.

Vulnerability Details

- The vulnerability is present in the `createNewDAOMembership` function where it has no checks to enforce a max cap on the allowed members for a sponsored DAO, allowing any arbitrary set members crossing the cap, i.e. `19825 / 7 tiers`.
- The `createNewDAOMembership` allows a user to create a DAO of 3 types, one type of the DAO is sponsored type DAO. The function takes the DAO config and tiers related config, where DAO config has a arbitrary member: `maxMembers` where it can be set to any value by the caller, but in case of a sponsored DAO it should be validated against the specific value the protocol wants to ensure that the `maxMembers` to be exactly 19825.
- Due to no checks on max members the sponsored DAO can be created with `maxMembers` value more than 19825, thus violating the invariant.
- Also, there is no enforcement of that in case of `updateDAOMembership`.

Impact

- The sponsored DAO can be created with more than 19825 members / 7 tiers, which breaks the protocol invariant.
- Also, the members can be changed using `updateDAOMembership`.

Tools Used

Manual Review

Recommendations

Either ensure that maxMembers is always 19825 for sponsored DAO or just neglect it and compare the total amount of members with 19825.